Technical Explanation

This document describes technically *how* the script achieves its behavior.

1. Imports

- pandas is used for data reading and transformation.
- plotly.graph_objects builds the chart.
- plotly.offline.plot writes the result to HTML.

2. Month Labels

A dictionary maps month numbers to Spanish abbreviations.
This is used later in tick formatting.

3. Data Loading Functions

load_banxico(path)

- Reads a CSV with:
  - custom encoding
  - header rows skipped
- Renames the key rate column.
- Filters rows where the date begins with "01" (first day of each month).
- Returns a cleaned DataFrame.

load_fed(path)

- Simply loads the FED CSV as-is.

4. Dataset Merge

The script:

1. Joins FED and Banxico using:
2. fed.join(banxico, how="left")
3. Converts observation_date to a datetime object.
4. Sets it as the DataFrame index.
5. Selects only the two rate columns.

This produces a unified time-series table indexed by actual dates.

5. Tick Generation

format_ticks(index) extracts:

- Every second date → tick positions
- Spanish-formatted strings → tick text

This reduces crowding on the x-axis.

6. Plot Construction

1. A Plotly figure is created.
2. A loop adds two traces:
    - FED funds rate
    - Banxico target rate

Each trace:

- Displays a line, markers, and value labels.
- Includes a custom hover template.

7. Layout Customization

- Title, legend, fonts, background colors, and hover behavior are defined in update_layout.
- Both axes get grid lines for readability.
- X-axis uses custom tick spacing and labels.

8. Output to HTML

pyo.plot() writes the figure to **interest_rates.html** and opens it automatically.
This ensures compatibility when running the script as a .py file.